SELF-HEALING HAMMING DECODER USING FSM-BASED RECOVERY AND TRIPLE MODULAR REDUNDANCY

Abstract

Modern digital systems operating in harsh environments such as satellites, spacecraft, medical devices, and safety-critical embedded systems are susceptible to transient faults caused by radiation, electromagnetic interference, and hardware degradation. Even a single bit flip can lead to corrupted data, incorrect system behavior, or complete system failure.

This project presents a Self-Healing Hamming Decoder capable of detecting, locating, and correcting single-bit errors using Hamming (7,4) error correction techniques. To improve system reliability, additional fault-tolerant mechanisms including One-Hot Finite State Machine (FSM) control, Illegal State Detection, Automatic FSM Recovery, and Triple Modular Redundancy (TMR) were incorporated.

An interactive Python-based demonstration interface was also developed to visualize fault injection, syndrome generation, error localization, correction, and successful data recovery. The system was implemented in Verilog HDL and verified through simulation using EDA Playground and EPWave.

The results demonstrate successful detection and correction of single-bit faults while maintaining system reliability under fault conditions.

---

1. Introduction

Digital communication systems frequently encounter errors during data transmission. In environments exposed to radiation or electrical noise, transmitted bits may unintentionally change from logic 0 to logic 1 or vice versa. Such faults, known as soft errors, can significantly impact system reliability.

Traditional error correction methods allow systems to detect and correct these faults before they propagate through the system. Among these methods, Hamming Code remains one of the most widely used techniques due to its simplicity and effectiveness.

This project extends conventional Hamming error correction by introducing self-healing capabilities through state recovery mechanisms and fault-tolerant architectural enhancements.

---

2. Problem Statement

Deep-space communication systems operate in highly radiation-intensive environments where transient faults frequently occur. A single radiation-induced bit flip may corrupt transmitted information and potentially compromise mission-critical operations.

The challenge is to design a fault-tolerant system capable of:

- Detecting transmission errors
- Locating faulty bits
- Correcting corrupted data
- Recovering from invalid controller states
- Maintaining reliable operation under fault conditions

---

3. Objectives

The primary objectives of this project are:

- Implement Hamming (7,4) encoding and decoding
- Detect and locate single-bit transmission errors
- Automatically correct corrupted data
- Design a One-Hot FSM controller
- Detect illegal FSM states
- Recover automatically from invalid states
- Implement Triple Modular Redundancy (TMR)
- Validate operation through waveform simulation
- Develop an interactive demonstration interface

---

4. Methodology

The proposed system consists of several interconnected modules.

4.1 Hamming Encoder

The Hamming Encoder converts a 4-bit input into a 7-bit codeword by generating parity bits.

The parity bits are calculated using XOR operations and positioned according to Hamming code conventions.

4.2 Fault Injector

To simulate real-world transmission faults, a fault injector deliberately flips a selected bit in the encoded codeword.

This module emulates radiation-induced transient faults commonly observed in aerospace systems.

4.3 Syndrome Generator

The syndrome generator evaluates parity relationships in the received codeword.

The generated syndrome uniquely identifies whether an error exists and, if present, indicates the position of the corrupted bit.

4.4 Error Locator

The syndrome value is directly mapped to the location of the faulty bit.

This allows the decoder to determine which position requires correction.

4.5 Correction Unit

The correction unit inverts the erroneous bit identified by the error locator.

The corrected codeword is then used to recover the original transmitted data.

4.6 FSM Controller

A One-Hot FSM Controller manages the operational states of the decoder.

Only one state remains active at any given time, simplifying state transitions and improving reliability.

4.7 Illegal State Detector

The illegal state detector continuously monitors the FSM state vector.

If multiple states become active simultaneously or no valid state exists, the detector raises an error flag.

4.8 Auto-Recovery FSM

Upon detection of an illegal state, the FSM automatically transitions to a predefined safe state.

This self-healing mechanism prevents system lockup and improves fault tolerance.

4.9 Triple Modular Redundancy (TMR)

Three identical FSM instances operate in parallel.

A majority voting mechanism determines the final output state.

This allows the system to tolerate faults affecting a single FSM instance without compromising overall functionality.

---

5. System Architecture

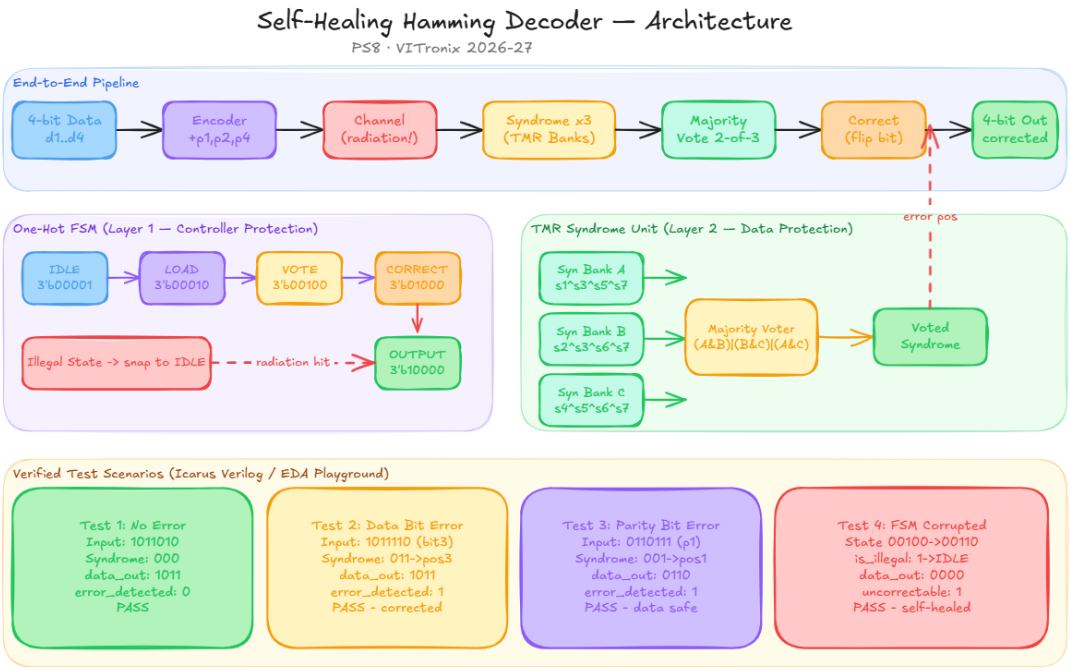

Figure 1. Proposed architecture of the Self-Healing Hamming Decoder integrating Hamming error correction, FSM-based recovery, and Triple Modular Redundancy.

---

6. Implementation

The hardware design was implemented using Verilog HDL.

The following modules were developed:

- Hamming Encoder
- Syndrome Generator
- Error Locator
- Correction Unit
- Self-Healing Hamming Decoder
- FSM Controller
- Illegal State Detector

- Auto-Recovery FSM
- TMR Majority Voter
- Complete Integrated Demonstration Module

Additionally, a Python GUI was developed to provide a user-friendly interface for demonstrating system functionality.

---

7. Simulation and Verification

Simulation was performed using EDA Playground.

The design was verified through multiple fault-injection test cases.

The following signals were observed:

- Input Data
- Encoded Codeword
- Corrupted Codeword
- Syndrome
- Error Position
- Corrected Codeword
- Recovered Data

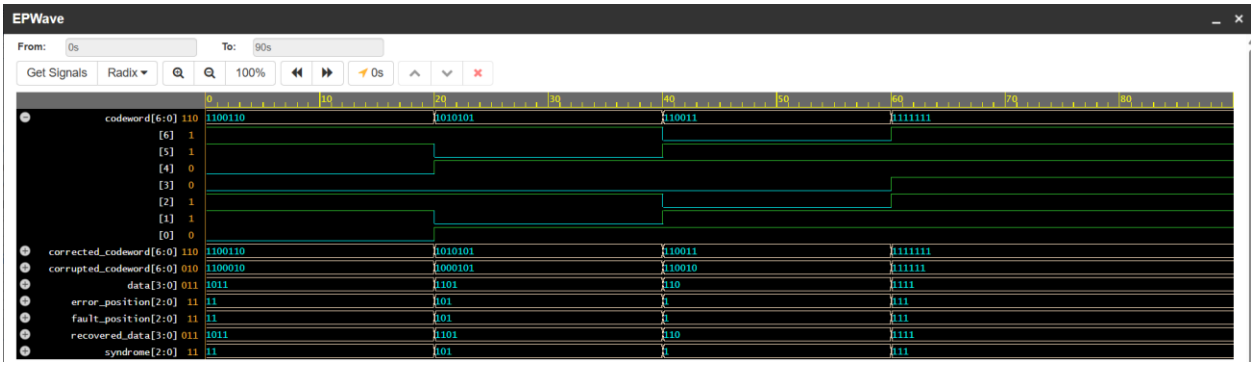

Figure 2. Waveform verification showing successful detection, localization, correction, and recovery of corrupted data.

---

8. Interactive Python Demonstration

A Python-based GUI was developed using Tkinter.

The interface allows users to:

- Enter 4-bit input data
- Inject random faults
- Observe syndrome generation
- Visualize error correction
- Verify successful data recovery

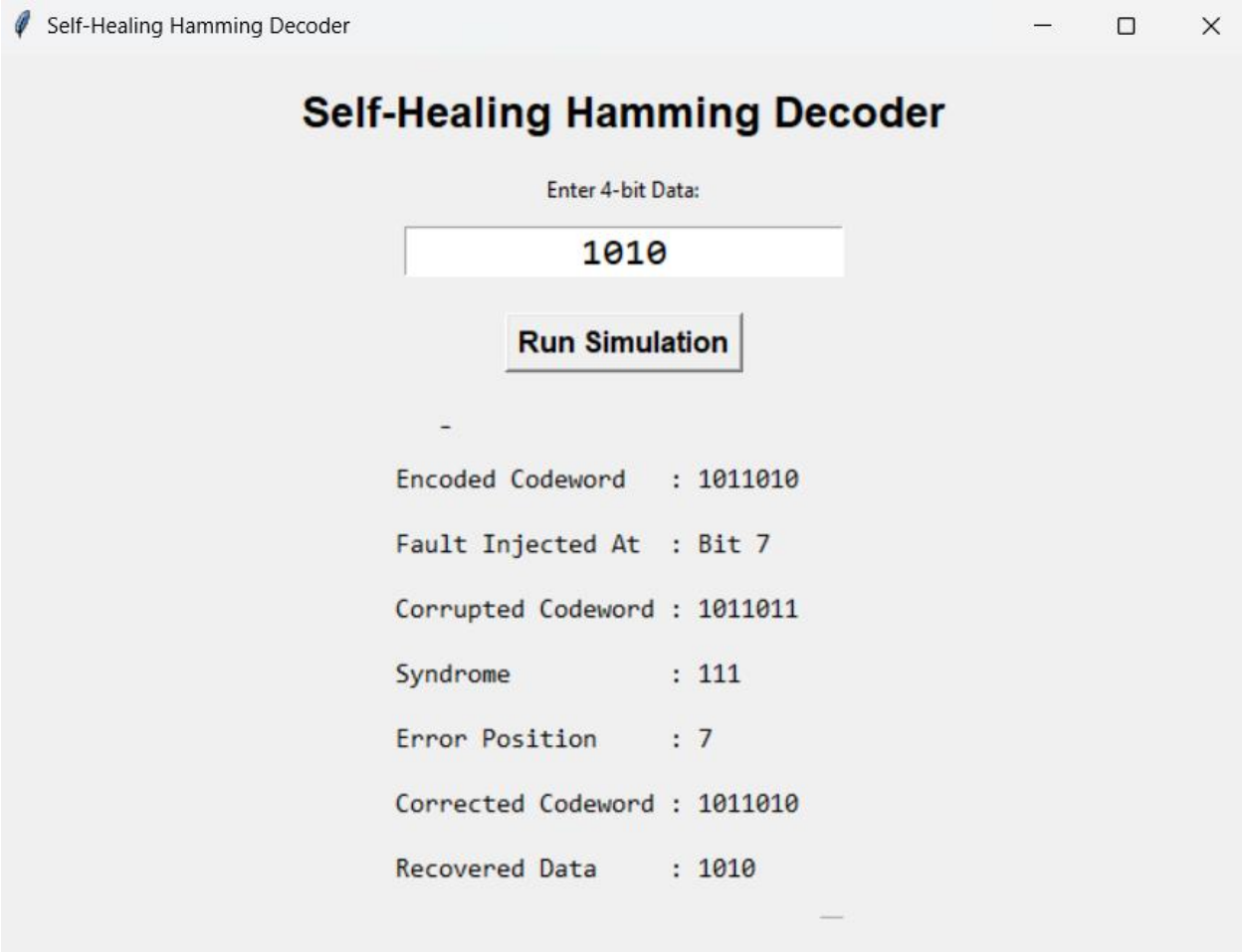

Figure 3. Interactive Python GUI demonstrating fault injection and self-healing recovery.

---

9. Results

The proposed system successfully:

- Detected single-bit transmission errors
- Located faulty bit positions
- Corrected corrupted codewords
- Recovered original data

- Detected illegal FSM states
- Recovered automatically from invalid states
- Maintained reliability using TMR

The waveform analysis confirmed successful operation under multiple fault scenarios.

---

10. Advantages

The proposed design offers several advantages:

- High reliability
- Automatic error correction
- Self-healing operation
- Enhanced fault tolerance
- Radiation resilience
- Modular architecture
- Easy scalability
- Educational visualization through GUI

---

11. Applications

Potential applications include:

- Satellite communication systems
- Aerospace electronics
- Medical devices
- Embedded systems
- Industrial control systems
- Safety-critical digital systems
- Radiation-prone computing environments

---

12. Future Scope

Future enhancements may include:

- FPGA implementation
- SECDED (Single Error Correction Double Error Detection)
- Multi-bit error correction

- Real-time fault injection
- Hardware-software co-simulation
- Radiation-aware fault modeling
- Advanced visualization dashboards

---

13. Conclusion

This project successfully demonstrates the design and implementation of a Self-Healing Hamming Decoder capable of detecting, locating, and correcting single-bit transmission errors. By integrating FSM-based recovery mechanisms and Triple Modular Redundancy, the reliability and resilience of the system were significantly enhanced.

Simulation results verified the effectiveness of the proposed architecture, while the Python-based demonstration interface provided an intuitive visualization of system operation. The project highlights the importance of fault-tolerant design techniques in modern digital systems and provides a strong foundation for future research in reliable computing architectures.

References

1. Richard W. Hamming, “Error Detecting and Error Correcting Codes,” Bell System Technical Journal, 1950.
2. M. Mano and M. Ciletti, Digital Design, Pearson Education.
3. Stephen Brown and Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design.
4. IEEE Standard Verilog Hardware Description Language Documentation.
5. EDA Playground Documentation.